NP-completeness

Pritha Banerjee

University of Calcutta

banerjee.pritha74@gmail.com

November 20, 2025

Overview

1. Complexity Class
2. References

Efficient Algorithm

- Algorithm designers observed that there are two very general classes of **combinatorial problems**:
  - those that can be solved by an intelligent search process
  - those that involve simple brute-force search.
- Since most combinatorial problems involve choosing from an exponential set of possibilities, the key distinguishing feature in most cases was whether there existed a **polynomial time algorithm** for solving the problem
- An algorithm is said to **run in polynomial time** if its **worst-case running time is** $O(n^c)$, where c is a nonnegative constant.
- A computational problem is said to be solved **efficiently** if it is **solvable in polynomial time**. Higher worst-case running times, such as 2^n, $n!$, and n^n are not polynomial time.
- Exponential time algorithms, such as those running in 2^n time, are certainly not efficient, assuming that n is sufficiently large.

Hard Problems

- In 60's, a list of problems came up which were **"hard"** in the sense that **no known efficient algorithmic solutions existed** for these problems.
- **A belief: hard problems** turned out to be **equivalent**, in the sense that if **any one of them** could be solved in **polynomial time**, then all of them **could be solved in polynomial time**
- The mathematical theory, developed by **Richard Karp and Stephen Cook**, gave the notions of **P, NP, and NP-completeness**; thousands of problems were identified as being in equivalence class.
- **It is widely believed that none of them can be solved in polynomial time, but there is no proof of this fact.**
- The **biggest open problems in computer science: Is P = NP?**

Complexity Class	Examples
Polynomial Time	Minimum Spanning Trees, Shortest Paths, Chain Matrix Multiplication, LCS, Stable Marriage, Maximum Matching Network Flows, Minimum Cut
Equivalent (Believed Hard)	Vertex Cover, Hamiltonian Cycle Boolean Satisfiability, Set Cover, Clique Cover Clique, Independent Set, Graph Coloring Hitting Set, Feedback Vertex Set

Objective of the Complexity theory

- Here, we **do not prove** that a **problem can be solved efficiently** by **presenting an algorithm** for it.
- Instead, we try to show that **a problem cannot be solved efficiently**.
- How to do this?

Assumption: The input to the problems as a **string over some alphabet** that has a **constant number of characters** (e.g., a binary bit string or a Unicode encoding).

Decision Problems and Languages

To analyze the **"hardness"** of **optimization problems** we consider a simpler version of it, called **decision problems**

- A problem is called a **decision problem** if its output is a simple **"yes" or "no"** . (or you may think of this as True/False, 0/1, accept/reject).
- If for decision problem of the optimization roblem there is no efficient solution (hard), the optimizatin problem must be as hard as the decision problem.
- **Example Decision Problem:** Minimum spanning tree problem is an optimization problem. The decision problem for it is: **Given a weighted graph G and an integer k, does G have a spanning tree whose weight is at most k ?**
- **Language recognition problem:** $MST = \{(G, k) | G \text{ has a minimum spanning tree of weight at most } k\}$

Decision Problems and Languages

$MST = \{(G, k) | G \text{ has a minimum spanning tree of weight at most } k\}$

- (G, k) is a reasonable encoding of the pair G and k as a string.
- **Solving a decision problem:** when presented with a specific input string $x = (G, k)$, the algorithm would answer **"yes"** if $x \in MST$, that is, if G has a spanning tree of weight at most k, and **"no"** otherwise.
- if "yes", algorithm **accepts** the input, else it **rejects** the input; decision problem is thus language recognition problem
- **How to determine if $x \in MST$:** decode x as G and k; feed them into any efficient minimum spanning tree algorithm (Kruskal's, say); if the final cost of the spanning tree is at most k, we accept x and otherwise we reject it.

Complexity Class : P

- **Definition:** P is the set of all languages (i.e., decision problems) for which membership can be determined in (worst case) polynomial time.
- P corresponds to **set of all decisions problems** that can be **solved efficiently**, that is, in **polynomial time**; P is a **set of languages**.
- A set of languages that is defined in terms of **how hard it is to determine membership** is called a complexity class; P is a complexity class.
- Since Kruskal's algorithm runs in polynomial time, we have $MST \in P$. So far all problems you studied are in class P.

Polynomial time verification and certificates

Consider Hamiltonian Cycle problem

- $HC = \{G | G$ has a simple cycle that visits every vertex of G$\}$
- Is $HC \in P$? No one knows the answer for sure, but it is **conjectured** that it is not. (In fact, HC is NP-complete, another complexity class)
- **Polynomial time verification:** Many language recognition problems that may be hard to solve, but they can be **verified** in polynomial time whether the string is in the language.
- **Example of verification:** For HC, if one wants to show the existence of a HC in G, he(she) can give **vertices in the order that they appear along the cycle**. The **verification algorithm** with this input (called, **certificate**) would be to check that it visits all the vertices exactly once. This is $O(n)$ time, polynomial time verification algorithm for HC.

Polynomial time verification and certificates

- A **certificate** is a piece of information which allows us to verify that a given string is in a language in polynomial time.
- More formally, given a language L, and given $x \in L$, a **verification algorithm** is an algorithm which, given x and a string y called the **certificate**, can verify that x is in the language L using this certificate as help. If x is not in L then there is nothing to verify. If there exists a verification algorithm that runs in polynomial time, we say that L can be verified in polynomial time.
- **Not possible to have polynomial time verification algorithm for the following**
  - $UHC = \{G | G$ has a unique simple cycle that visits every vertex of G$\}$
  - $NHC = \{G | G$ has no simple cycle that visits every vertex of G$\}$

Complexity Class: NP

NP stands for Non-deterministic Polynomial

- **Definition:** NP is the set of all languages that can be verified in polynomial time.
- $P \subseteq NP$, since it is possible to design polynomial time verification algorithm for all problems in P

Reduction

- H is known hard problem and complexity of U is unknown
- Polynomial time Karp reduction $H \leq_P U$

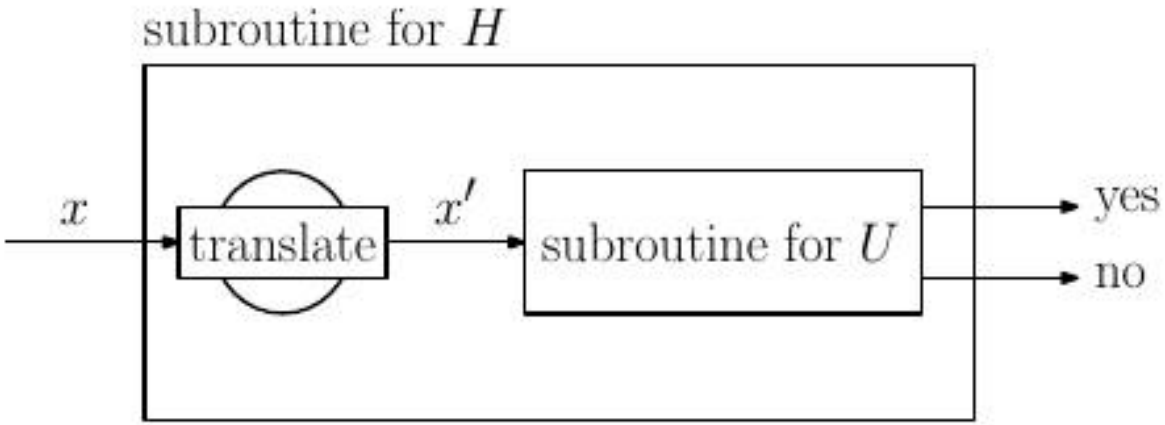
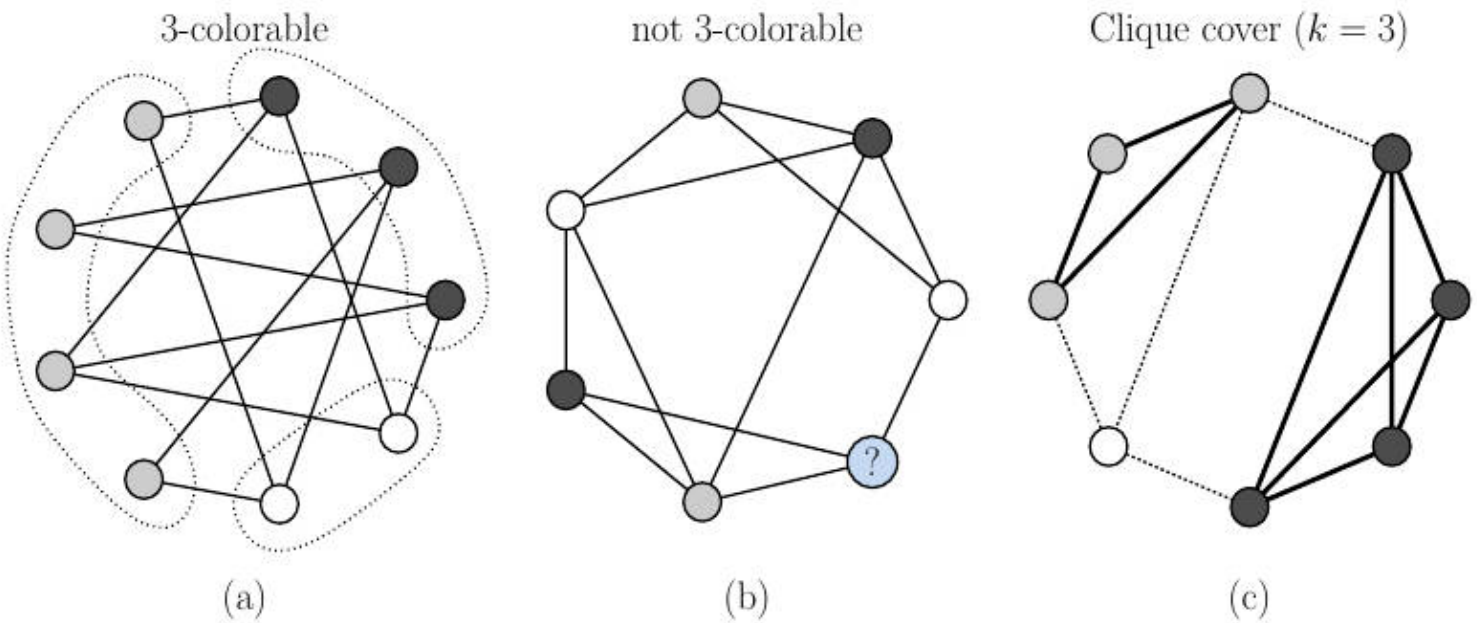


Fig. 77: Reducing H to U.

3-Colorability and Clique Cover

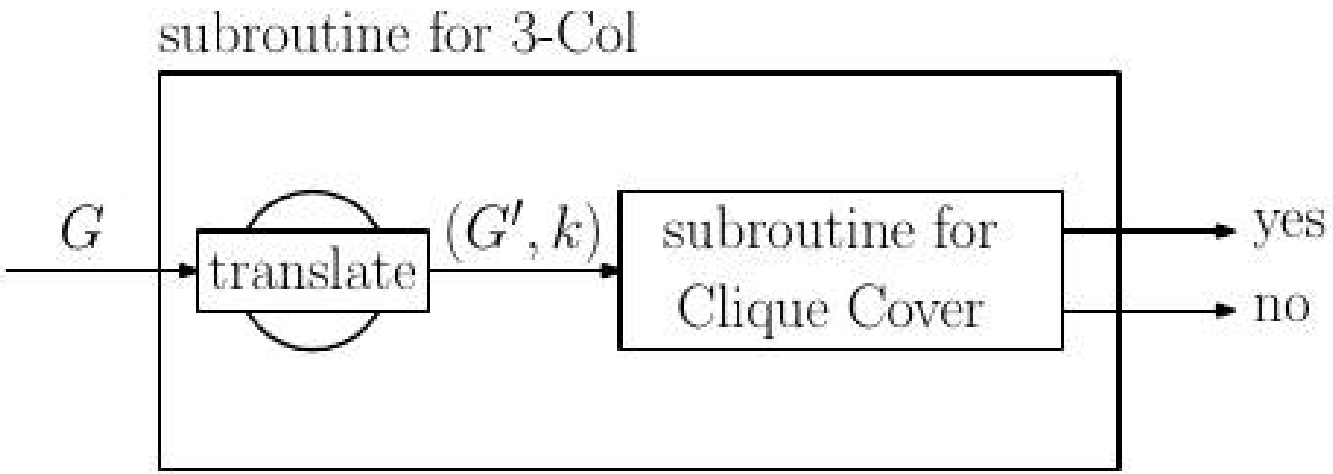
- **3-Col:** Given a graph G, can each of its vertices be labeled with one of three different "colors", such that no two adjacent vertices have the same label
- **Clique Cover (CCov):** Given a graph $G = (V, E)$ and an integer k, can we partition the vertex set into k subsets of vertices $V_1, \cdots, V_k$ such that each V_i is a clique of G.



3-Colorability to Clique Cover: Reduction

Convert an instance of 3Col problem G into an equivalent instance of the Clique cover problem $(G', 3)$

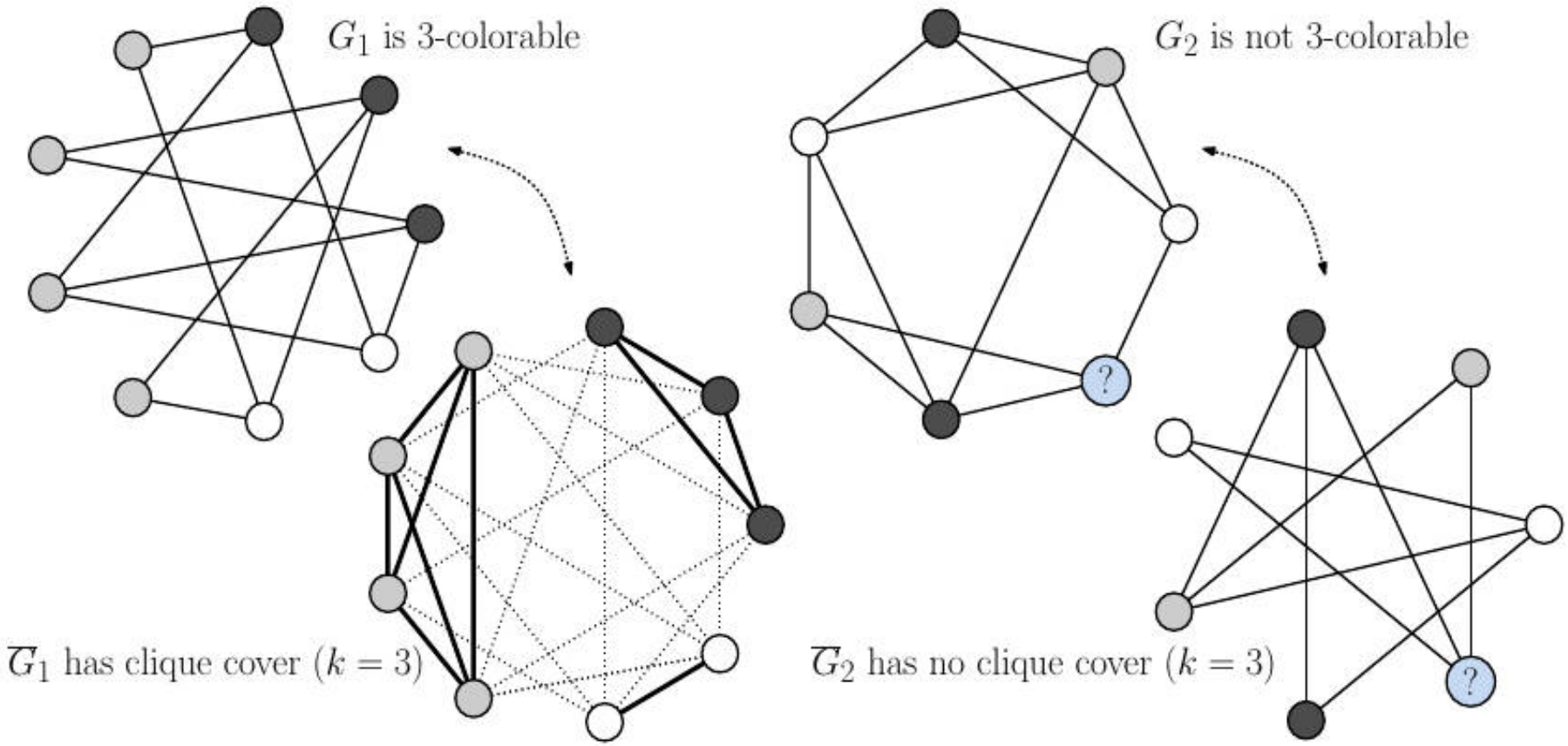
- **translation function** $f(x)$**:** Compute **complement graph** $G' = (V, E')$ of $G = (V, E)$. E' is set of all edges (u, v) that are not in G, i.e, $(u, v) \notin E$. Construction of G' is $O(n + m)$, considering adjacency list representation of graph, which is polynomial
- **Claim:** A graph $G = (V, E)$ is 3-colorable **if and only if** its complement $G' = (V, E')$ has a clique-cover of size 3. In other words, $G \in 3Col \iff (G', 3) \in CCov$



3-Colorability to Clique Cover: Reduction Proof

- $\Longrightarrow$: If G 3-colorable, then let V_1, V_2, V_3 be the three color classes. We claim that this is a clique cover of size 3 for G', since if u and v are distinct vertices in V_i, then $(u, v) \notin E$ (since adjacent vertices cannot have the same color). Thus $(u, v) \in E'$; Thus every pair of distinct vertices in V_i are adjacent in G'.
- $\Longleftarrow$: Suppose G' has a clique cover of size 3, denoted V_1, V_2, V_3. For $i \in 1, 2, 3$ give the vertices of V_i color i. This is a legal coloring for G, since if distinct vertices u and v are both in V_i, then $(u, v) \in E'$ (since they are in a common clique), implying that $(u, v) \notin E$. Hence, two vertices with the same color are not adjacent.

3-Colorability to Clique Cover: Reduction Proof



Polynomial time reduction

- **Definition:** A language (i.e. decision problem) L_1 is **polynomial-time reducible** to language L_2 (written $L_1 \leq_P L_2$) if there is a **polynomial time computable function** f , such that for all $x, x \in L_1$ if and only if $f(x) \in L_2$.
- Example: $3Col \leq_P CCov$, and in particular, $f(G) = (G', 3)$ in $O(n^2)$ (i.e. polynomial) time (e.g. flip 0's and 1's in adjacency matrix representation of graph G to get G')
- **Intuition to determine complexity of a "hard" problem:** since L_1 is **not likely to be solvable in polynomial time** , then L_2 is **also not likely to be solvable in polynomial time**.

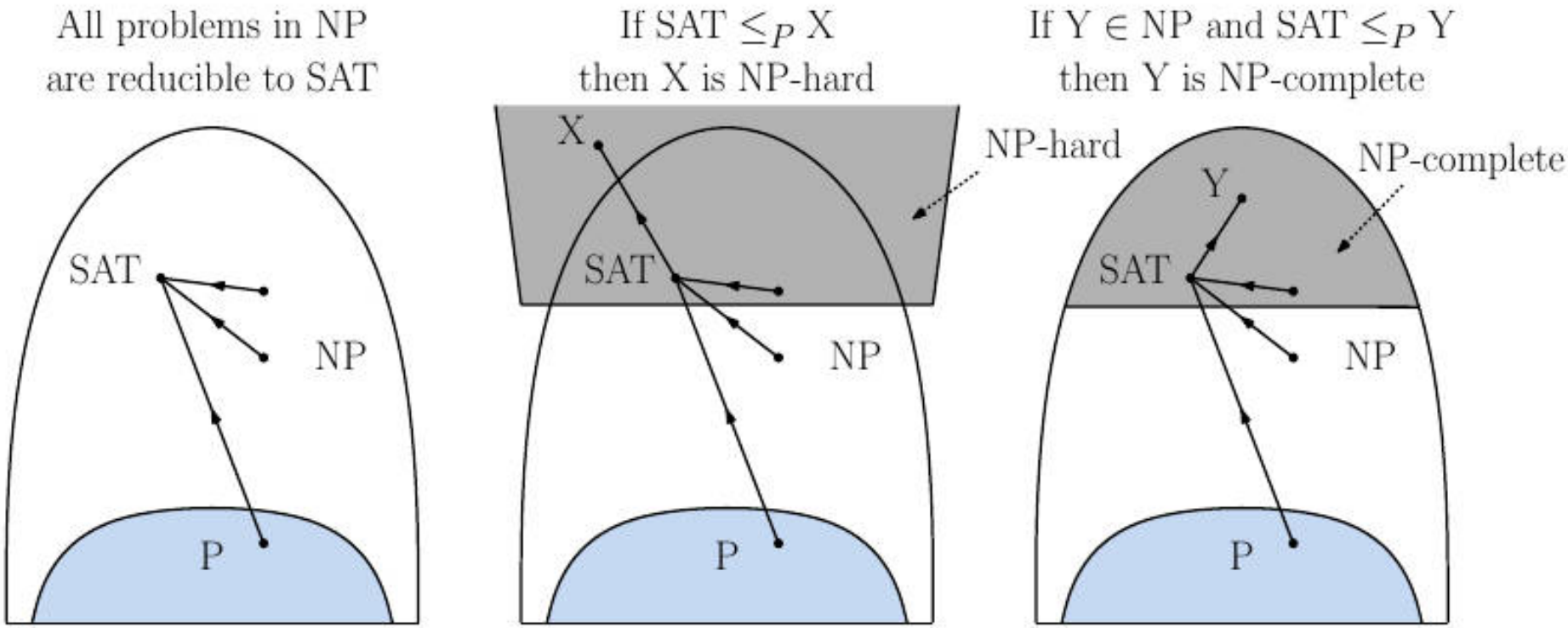
Three important Lemma and NP completeness

- **Lemma 1:** If $L_1 \leq_P L_2$ and $L_2 \in P$ then $L_1 \in P$.
- **Lemma 2:** If $L_1 \leq_P L_2$ and $L_1 \notin P$ then $L_2 \notin P$.
- **Lemma 3:** If $L_1 \leq_P L_2$ and $L_2 \leq_P L_3$ then $L_1 \leq_P L_3$; because the composition of two polynomials is a polynomial.
- **NP-completeness:** The set of NP-complete problems are all problems in the complexity class NP, for which it is known that if any one is solvable in polynomial time, then they all are, and conversely, if any one is not solvable in polynomial time, then none are.
- **Definition:** A language L is **NP-hard** if $L' \leq_P L$, for all $L' \in NP$. (Note that L does not need to be in NP.)
- **Definition:** A language L is **NP-complete** if:
  - $L \in NP$ (that is, it can be verified in polymomial time), and
  - L is **NP-hard** (every problem in NP is polynomially reducible to it).

How to prove a problem is NP-Complete?

- **Lemma 4:** L is NP-complete if
  - $L \in NP$ and
  - $L' \leq_P L$ for **some known NP-complete** problem L' .
- Lemma 4 is true because of transitivity of Lemma 3.
- Cook showed that there is one problem called SAT (short for boolean satisfiability) that is NP-complete and all problems in NP are reducible to SAT.
- To prove a second problem L is NP-complete, show that our problem $L \in NP$ and then show that $SAT \leq_P L$ (or generally some known NPC problem) is polynomial time reducible to our problem L.

Structure of NP completeness and Reduction



Boolen Satisfiability Problem

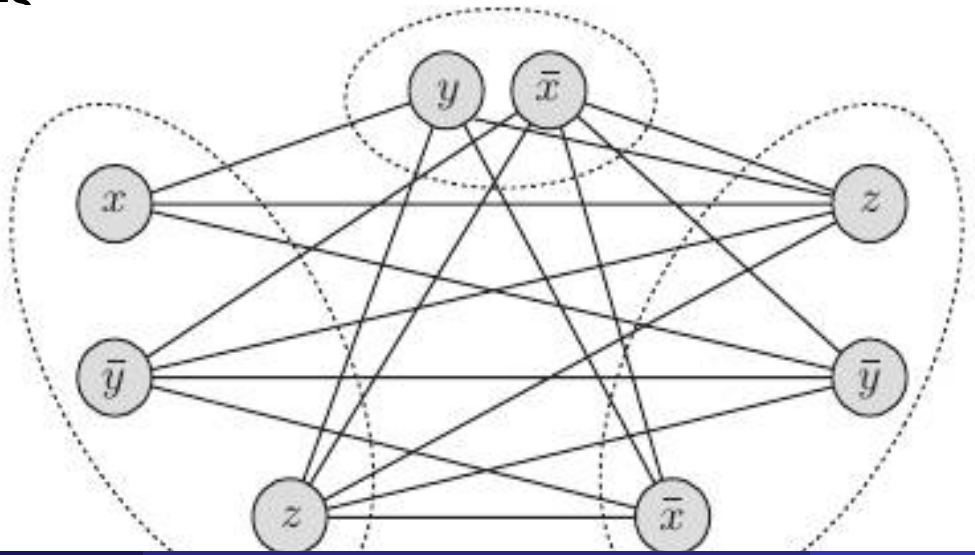
- A boolean formula consists of variables (say x, y, and z) and the logical operations not ($\bar{x}$), and ($x \wedge y$), and or ($x \vee y$).
- Given a boolean formula, we say that it is **satisfiable** if there is a **way to assign truth values** (0 or 1) to the variables such that it **evaluates to** 1.
- **Example 1:** $F_1(x, y, z) = (x \wedge (y \vee \bar{z})) \wedge ((\bar{y} \wedge \bar{z}) \vee \bar{x})$ is **satisfiable** for $x = 1$ and $y = z = 0$
- **Example 2:** $F_2(x, y) = (\bar{z} \vee x) \wedge (z \vee y) \wedge (\bar{x} \wedge \bar{y})$ is **not satisfiable** as every possible assignment of x, y, z evaluates it to 0.
- **Cook's Theorem:** $SAT \in NP$ and SAT is NP Hard. Thus, SAT is NP-Complete
- **literal:** A **literal** is either a variable or its negation, that is, x or $\bar{x}$.
- **3-Conjunctive Normal Form (3-CNF):** A boolean formula is said to be in **3-CNF** if it is the **boolean and of clauses** where **each clause is the boolean or** of **exactly three literals**.
- Example 3-CNF: $(x_1 \vee x_2 \vee x_3) \wedge (x_1 \vee x_3 \vee x_4) \wedge (x_2 \vee x_3 \vee x_4)$ is in 3-CNF form.

Example: To prove Clique problem is NP-Complete (1)

- Clique: Given an undirected graph $G = (V, E)$ and a positive integer k, does G contain a clique of size k? (recall that a clique in G of size k is a complete subgraph of G on k vertices).
- To prove Clique problem is NP Complete : 1) show $Clique \in NP$ 2) show $SAT \leq_P Clique$
- $Clique \in NP$: Given G, k, a certificate c a set of k vertices of G, consider all possible pairs of vertices of k to see if there exists an edge. The time complexity of this algorithm is $O(n^2)$. Thus, a polynomial time verification algorithm exist and $Clique \in NP$
- $SAT \leq_P Clique$: To show if $x \in SAT$ then $f(x) \in Clique$ and if $x \in Clique$ then $f(x) \in SAT$ (iff), where x is any input, for Clique it is $< G, k >$, for SAT it is boolean function F

Example: To prove Clique problem is NP-Complete (2)

- Given an instance of satisfiability $F = C_1 \wedge C_2 \wedge \cdots C_m$ with m clauses and n boolean variables $x_1, x_2, \cdots, x_n$. If there exists an assignment of x_i which evaluates to 1 then F is satisfiable.
- To show F is satisfiable iff G has a clique of size m
- **Reduction:** Construct a graph $G = (V, E)$, where V is the set of all occurrences of the $2n$ literals and $E = \{(x_i, x_j) | x_i \text{and} x_j$ are in two different clauses and$x_i \neq \bar{x}_j\}$ and let $k = m$
- **Example:** $F = (x \vee \bar{y} \vee z) \wedge (\bar{x} \vee y) \wedge (\bar{x} \vee \bar{y} \vee z)$, $m = 3$ clauses $n = 3$ variables



Example: To prove Clique problem is NP-Complete (3)

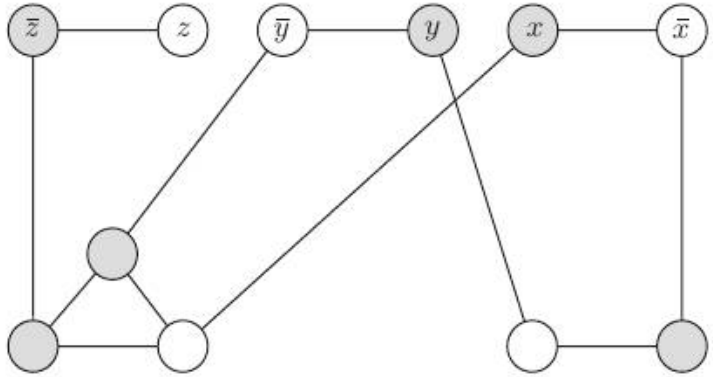
- Reduction Algorithm is polynomial time, since it involved creation of $2n$ vertices and atmost n^2 edges.
- A clique of size m corresponds to an assignment of true to m literals in m different clauses. An edge between two literals a and b means that there is no contradiction when both a and b are assigned the value **true**.
- $\Longleftarrow$: To show if $x \in SAT$ then $f(x) \in Clique$; It follows that if F is satisfiable there is a non-contradictory assignment of true to m literals in m different clauses which implies that those vertices and the edges between them in G are chosen, which is a clique of size m
- $\Longrightarrow$: to show if $x \in Clique$ then $f(x) \in SAT$; It follows that if there is a clique of size m in G, atleast one vertex from every clause will be assigned to true, thus, F is satisfiable.

Vertex cover is NP-complete (1)

Vertex cover problem (VC): Given an undirected graph $G = (V, E)$ and a positive integer k, is there a subset $C \subseteq V$ of size k such that each edge in E is incident to at least one vertex in C? a) Satisfiability (SAT) $\leq_P$ Vertex Cover (VC) b) $VC \in NP$

- Given an instance I of SAT, transform it into an instance I' of VC.
- Let I be the formula $f = C_1 \wedge C_2 \wedge \cdots \wedge C_m$ with m clauses and n boolean variables $x_1, x_2, \cdots, x_n$.
- **Construction:**
  - For each boolean variable x_i in f , G contains a pair of vertices x_i and $\bar{x}_i$ joined by an edge.
  - For each clause C_j containing n_j literals, G contains a clique C_j of size n_j .
  - For each vertex w in C_j , there is an edge connecting w to its corresponding literal in the vertex pairs (x_i, x_i) constructed in step 1. Call these edges **connection edges**.
  - Let $k = n + \sum_{j=1}^{m}(n_j - 1)$.
- Construction or translation function is polynomial $O(n + m)$ using adjacency list.

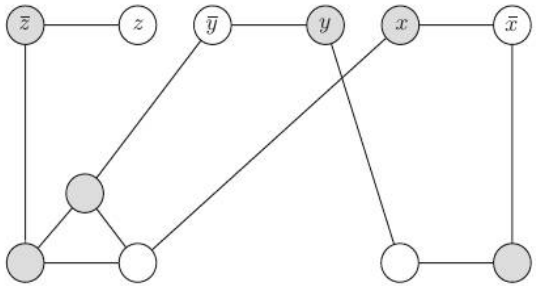
Vertex cover is NP-complete (2)



Let $f = (x \vee \bar{y} \vee \bar{z}) \wedge (\bar{x} \vee y)$ In I', $k = 3 + 2 + 1 = 6$. $x = \mathit{true}, y = \mathit{true}$ and $z = \mathit{false}$ satisfies f. Covering vertices shown shaded.

- To show: f is satisfiable if and only if the constructed G has a vertex cover of size k.
  - $\Longrightarrow$: If x_i is assigned true, add vertex x_i to the vertex cover; otherwise add $\bar{x}_i$ to the vertex cover.
  - Since f is satisfiable, in each clique C_j there is a vertex u whose corresponding literal v has been assigned the value true, and thus the connection edge (u, v) is covered.
  - Therefore, add the other $n_j - 1$ vertices in each clique C_j to the vertex cover. Thus, the size of the vertex cover is $k = n + \sum_{j=1}^{m}(n_j - 1)$.

Vertex cover is NP-complete (3)



Proof: $\Longleftarrow$:

- Suppose that the graph can be covered with k vertices. At least one vertex of each edge $(x_i, \bar{x}_i)$ must be in the cover. We are left with $k - n = \sum_{j=1}^{m}(n_j - 1)$ vertices.
- Any cover of a clique of size n_j must have exactly $n_j - 1$ vertices. So, in each clique, the cover must include all its vertices except the one that is incident to a connection edge that is covered by a vertex in some vertex pair $(x_i, \bar{x}_i)$.
- To see that f is satisfiable, for each vertex x_i, if it is in the cover then let $x_i = \mathit{true}$; otherwise (if $\bar{x}_i$ is in the cover), let $x_i = \mathit{false}$.
- Thus, in each clique, there must be one vertex which is connected to a vertex x_i or $\bar{x}_i$, which is assigned the value true since it is in the cover.
- It follows that each clause has at least one literal whose value is true, i.e., f is satisfiable.

Self- Study

Prove that the following problems are NP-complete

- Independent Set problem: show Vertex cover $\leq_P$ Independent set
- Hamiltonian Path
- Knapsack Problem
- Bin Packing
- Set cover
- Multiprocessor Scheduling
- Longest Path

References

- Lecture notes on Design and Analysis of Algorithm , David Mount, 2015, Lecture 21,22